DOC-TRIZ-001
v0.3.1-triz
2026-10-28

АНАЛИТИЧЕСКИЙ ОТЧЁТ / ТРИЗ-АНАЛИЗ АРХИТЕКТУРНОЙ ДОКУМЕНТАЦИИ / БИЛИНГВА RU+EN

# TRIZ-анализ документация CodeAuthorityFirst

Применение теории решения изобретательских задач к ARCHITECT.md, UI.md, ROADMAP.md, BACKLOG.md, ADR.md в направлении максимального понимания кода агентами и максимальной скорости UI.

**ИКР**

Документация сама себя объясняет LLM-агенту за один проход без дополнительных инструментов, при этом любой UI-бюджет находится в файле не далее 2 экранов от описания экрана.

**ПРИОРИТЕТ**

Агентопонятность > UI-скорость (UI-скорость как следствие). Полный прогон: ИКР → противоречия → матрица 39×39 → 40 приёмов → АРИЗ → контрольное решение.

| 5 | 15 | 12+ | 7 |
|---|---|---|---|
| документов переписано | инконсистентностей исправлено | приёмов из 40 применено | @invariant флагов добавлено |

SOURCE github.com/Reider85/OryaCodeAuthorityFirst
BRANCH main · analytics/
DATE 2026-10-28 UTC+3
PAGES 10 (cover + TOC + 8 chapters)

# Содержание

**Глава 1. Постановка задачи и методология ТРИЗ 3**

1.1 Ключевые находки дайджеста (входные данные для ТРИЗ) . . . 3

1.2 Почему приоритет «агенты > UI» обоснован . . . 4

**Глава 2. ИКР-формулировка для всей документации 4**

2.1 Декомпозиция ИКР на пять под-ИКР (по одному на файл) . . . 5

2.2 Почему «без дополнительных инструментов» — это не пассивность, а активное свойство . . . 5

**Глава 3. Сквозной ТРИЗ-анализ по пяти файлам 5**

3.1 ARCHITECT.md . . . 6

3.2 ROADMAP.md . . . 6

3.3 UI.md . . . 7

3.4 BACKLOG.md . . . 7

3.5 ADR.md . . . 8

**Глава 4. Сводная матрица противоречий 39×39 9**

**Глава 5. Подобранные приёмы из 40 — маппинг на артефакты 10**

**Глава 6. АРИЗ-шаги для двух ключевых проблем 12**

6.1 АРИЗ-прогон для проблемы №1: ARCHITECT.md = дубликат ROADMAP.md . . . 12

6.2 АРИЗ-прогон для проблемы №2: BACKLOG.md устаревшие пункты + дубликаты + битые ссылки . 14

**Глава 7. Резюме улучшений и trace-матрица 15**

7.1 Было → стало по 5 файлам . . . 15

7.2 Trace-матрица: Цель → ADR → Backlog → Код → UI . . . 16

**Глава 8. Ограничения, компромиссы и следующие шаги 17**

8.1 Что ТРИЗ не решил . . . 17

8.2 Компромиссы, принятые в переписанных файлах . . . . . 18

8.3 Следующие шаги для пользователя . . . . . 18

8.4 Итог . . . . . 19

# Глава 1. Постановка задачи и методология ТРИЗ

***EN summary.*** *Chapter 1 establishes the task context (5 markdown files from the Reider85/OryaCodeAuthorityFirst repository) and the chosen priority (agent code understanding takes precedence over UI speed — UI speed is treated as a downstream consequence). It also surfaces the four most consequential inconsistencies found in the source corpus, which the TRIZ pass must resolve.*

Задача, поставленная пользователем: применить ТРИЗ к пяти документам репозитория Reider85/OryaCodeAuthorityFirst — ARCHITECT.md, UI.md, ROADMAP.md, BACKLOG.md, ADR.md — в направлении двух целевых осей: максимальной скорости UI и максимального понимания кода LLM-агентами. По итогам уточняющих вопросов пользователь выбрал приоритет «агенты > UI»: агентопонятность — мета-цель, а скорость UI — её производная. Логика приоритета проста: если документация не объясняет код агенту, агент вносит регрессии в UI-скорость, поэтому первичное укрепление агентопонятности автоматически поднимает и UI-скорость в долгосрочной перспективе.

Метод ТРИЗ, применённый в этой работе, следует канонической последовательности Альтшуллера: (1) формулировка идеального конечного результата (ИКР); (2) выявление и формулировка технических противоречий в формате «если A, то ухудшается B»; (3) обращение к матрице 39×39 для подбора релевантных изобретательских приёмов; (4) применение подобранных приёмов из 40; (5) полный прогон АРИЗ-85-В для двух наиболее острых проблем; (6) контрольное решение — проверка, что предложенные улучшения действительно устраняют противоречие, а не создают новое. Глубина применения — полная: для каждого файла отдельно формулируются ИКР, противоречия и приёмы.

## 1.1 Ключевые находки дайджеста (входные данные для ТРИЗ)

Перед применением ТРИЗ был проведён исчерпывающий дайджест всех пяти файлов (с привлечением вспомогательного агента, который прочитал каждый файл целиком и зафиксировал структуру, таблицы, кросс-ссылки, UI-бюджеты и инконсистентности). Из дайджеста выделены четыре наиболее критичных дефекта, формирующих входные данные для ТРИЗ-анализа:

Во-первых, файл ARCHITECT.md оказался байт-идентичным дубликатом ROADMAP.md — оба файла содержат H1 «# CodeAuthorityFirst — ROADMAP.md» и идентичное тело. Это значит, что архитектурного документа в репозитории фактически нет, а все ссылки на ARCHITECT.md из ADR.md (их четыре: §3.3.2 capability matrix, §6.5 fail-safe, §5.2 config, §3.3.2 снова) ведут в никуда или в семантически нерелевантные секции. Capability matrix, явно названная «first-class artifact» в ADR-009, физически отсутствует в репозитории.

Во-вторых, BACKLOG.md трижды ссылается на ROADMAP.md §6.10 — несуществующую секцию (§6 FUTURE содержит только подразделы 6.1–6.9). Реальные четыре критерия промоушена находятся в §6.9. Каждое обсуждение promotion в BACKLOG отправляет читателя (или агента) на несуществующий якорь.

В-третьих, в ADR-012 (и параллельно в ROADMAP §5.1) Tier-3 список агентов содержит 15 названий, но помечен как «(12 agents)». Арифметика «25 explicit» работает только если Tier 3 = 12; с фактическими 15 суммарное число — 28. Это расхождение просачивается в UI.md §7.2, где «25 available» и «21 available» соседствуют в одном экране.

В-четвёртых, UI.md §1.1 и ROADMAP §2.1/§4.1 дают противоречивые фазовые назначения: Dashboard в UI.md указан как ALPHA, а в ROADMAP — как MVP; File tour в UI.md указан как PROD, а в ROADMAP — как BETA. Два экрана рассинхронизированы ровно на одну фазу каждый. Помимо этих четырёх, дайджест выявил ещё 11 более мелких дефектов (устаревшие v0.2 строки, китайские символы в русских текстах, тип в K-012, арифметическая ошибка в статистике BACKLOG и т.д.) — все они учтены в переписанных версиях файлов.

### 1.2 Почему приоритет «агенты > UI» обоснован

Выбор приоритета не произволен. Скорость UI — это свойство, которое агентопонятность усиливает, а не подавляет. Когда документация объясняет агенту, какие UI-бюджеты действуют на каком экране и какие инварианты нельзя нарушать, агент вносит изменения, уважающие эти бюджеты. Когда документация не объясняет этого, агент вносит «оптимизации», которые ломают инварианты и регрессируют UI-скорость. Таким образом, инвестирование в агентопонятность — это инвестирование в UI-скорость с отложенным выигрышем. Обратное неверно: инвестирование в UI-скорость без агентопонятности даёт временный пик, который агент тут же разрушает в следующей итерации.

Поэтому в переписанных файлах UI-бюджеты не выдвигаются вперёд как самостоятельная цель — они встроены в более широкий каркас инвариантов (YAML front-matter в каждом файле, @invariant теги в каждом ADR, явные trace-ссылки от ADR к коду и UI-секциям). Это и есть применение ТРИЗ-приёма №17 «переход в другое измерение»: скорость UI теперь не «отдельная глава», а свойство, проявляющееся во всех главах через инварианты.

## Глава 2. ИКР-формулировка для всей документации

***EN summary.*** *Chapter 2 formulates the Ideal Final Result (ИКР) for the documentation corpus as a whole, then decomposes it into five per-file sub-ИКРs. The corpus-level ИКР states that documentation must self-explain to an LLM agent in one pass without auxiliary tools, and any UI budget must be no further than two screens from its screen description.*

ИКР (идеальный конечный результат) для всей документации формулируется так: документация сама себя объясняет LLM-агенту за один проход без дополнительных инструментов, при этом любой UI-бюджет находится в файле не далее двух экранов от описания экрана, к которому он применяется. Формулировка намеренно строгая: «без дополнительных инструментов» означает, что агент не должен запускать grep, ripgrep, lex-парсер или второй LLM, чтобы понять, какие ADR затрагивает конкретный backlog-пункт. «Не далее двух экранов» — это конкретное измеримое ограничение, которое можно проверить, открыв любой экран в UI.md и убедившись, что его performance budget находится в той же секции.

Ключевое слово в формулировке — «сама себя». Это значит, что документация не должна делегировать объяснение внешним артефактам: ни `caf doc-check` (которого ещё нет), ни внешнему README, ни сопровождающему туториалу. Внутри пяти .md файлов должен быть достаточный контекст для любого разумного вопроса агента о коде. Если контекста нет — это противоречие, которое ТРИЗ должен устранить.

## 2.1 Декомпозиция ИКР на пять под-ИКР (по одному на файл)

**Под-ИКР для ARCHITECT.md.** Файл самодостаточен: capability matrix существует внутри файла как таблица (не как «see §3.3.2» в никуда); fail-safe контракт описан прямо в §2.5 Risks как вложенный блок; конфигурационная схема — в §7.4 Backwards compatibility. Любой агент, открывающий ARCHITECT.md, находит в первом экране YAML front-matter со списком инвариантов, owned_contracts и agent_routing — т.е. маршруты «если задача X, смотри §Y».

**Под-ИКР для ROADMAP.md.** Файл дедуплицирован (больше не байт-идентичен ARCHITECT.md), каждая фаза имеет trace к ADR (через Depends_on и Depends_on поля в YAML front-matter) и к BACKLOG items (через явные ссылки в §2.1–§6.1 scope). Четыре критерия промоушена находятся в §6.9 (а не §6.10), и BACKLOG ссылается именно туда.

**Под-ИКР для UI.md.** Каждый экран несёт свой performance budget в той же секции (§3.6 для dashboard, §4.7 для review, §5.5 для decisions, §6.5 для heatmap, §7.5 для adapters, §8.4 для file tour). Бюджеты снабжены @invariant тегами (INV-UI-001 … INV-UI-007). Skeleton/SWR паттерн объявлен дефолтным для всех async-загрузок. Phase-конфликты (Dashboard, File tour) разрешены в сторону ROADMAP как канонического источника фазовых назначений.

**Под-ИКР для BACKLOG.md.** Каждая идея содержит поле agent_hint с ссылкой на ADR и код-контракт. Устаревшие пункты (B-004, B-005, B-006, B-008, B-009, I-007, J-003) помечены done. Кросс-тематические дубликаты (K-003=B-001, K-004=B-002, K-005=B-003, K-006=D-006) явно отмечены как дубликаты. Статистика пересчитана и арифметически согласована. Трилингвальные слипы (китайские символы в D-006 «语音» и D-008 «合成») заменены на корректные русские эквиваленты.

**Под-ИКР для ADR.md.** Каждый ADR имеет @invariant тег в поле Trigger to reconsider. Четыре битые ссылки на ARCHITECT.md (§3.3.2, §6.5, §5.2) исправлены на конкретные секции нового ARCHITECT.md. Tier-3 miscount (15 vs 12) reconciled — OryaAgent убран из Tier 3 (это in-house consumer, описанный отдельно в ARCHITECT §6.5), Augment переименован единообразно. Future-ADR таблица пополнена ADR-029 «caf invariants check» — предложение pre-commit hook для enforcement @invariant тегов.

## 2.2 Почему «без дополнительных инструментов» — это не пассивность, а активное свойство

Формулировка «без дополнительных инструментов» — критическая. В текущем (до-TRIZ) состоянии корпуса, чтобы понять, какие ADR затрагивает конкретный backlog-пункт (например, BACKLOG A-005 — memory-mapped sessions index), агент должен: открыть BACKLOG.md, прочитать пункт A-005, заметить фразу «конфликтует с ADR-001», открыть ADR.md, найти ADR-001, прочитать контекст и выбранное решение, вернуться в BACKLOG.md, прочитать Sketch и Dependencies. Это пять шагов вместо одного. После TRIZ-прохода в BACKLOG.md каждый пункт содержит YAML-поле agent_hint с прямыми ссылками: «agent_hint: Затрагивает ADR-001, INV-ADR-001 (.caf/ size budget). Код: internal/caf/sessions/index.go». Один шаг вместо пяти. Это и есть реализация под-ИКР.

# Глава 3. Сквозной ТРИЗ-анализ по пяти файлам

***EN summary.*** *Chapter 3 runs the full TRIZ analysis (contradictions → 39×39 matrix → inventive principles) for each of the five files separately. Each subsection covers one file: 2–3 contradictions, the matrix parameters in conflict, the recommended principles, and a brief ARIZ mini-step.*

В этой главе каждый из пяти файлов анализируется отдельно. Для каждого формулируются 2–3 ключевых технических противоречия в формате «если A улучшается, то B ухудшается», определяются конфликтующие параметры из 39 инженерных параметров Альтшуллера, по матрице 39×39 подбираются рекомендованные приёмы из 40, и приводится краткий АРИЗ-мини-прогон. Полный АРИЗ-85-В прогон для двух самых острых проблем вынесен в Главу 6.

## 3.1 ARCHITECT.md

**Противоречие 3.1.A — «уникальность контента vs количество файлов».** Если ARCHITECT.md остаётся байт-идентичным ROADMAP.md (исходное состояние), то два файла несут одинаковое содержание → нарушается DRY, но поддерживать один файл проще, чем два разных. Если же ARCHITECT.md получает уникальное архитектурное содержание (наше решение), то корпус богатее, но требует поддержания двух разных файлов с одинаковой структурой. Конфликтующие параметры: параметр 33 «удобство восприятия/поиска информации» (растёт при уникализации) vs параметр 36 «потери информации» (могут возникнуть при расхождении версий). Матрица рекомендует приёмы 7, 17, 32 (по паре 33×36). **Применённые приёмы:** #4 «асимметрия» — ARCHITECT и ROADMAP сохраняют ту же 12-секционную структуру, но содержание дифференцировано (асимметрия по содержанию при симметрии по структуре); #17 «переход в другое измерение» — добавлен YAML front-matter как отдельное машинно-читаемое измерение поверх прозы; #32 «оптические изменения» — цветные @invariant теги.

**Противоречие 3.1.B — «наличие capability matrix vs объём файла».** Если capability matrix встроена в ARCHITECT.md как таблица, файл растёт (~на 1 страницу), но агент находит её сразу. Если матрица вынесена в отдельный файл, файл остаётся компактным, но агент должен открыть второй файл. Параметры: 27 «длина движущегося объекта» (текст файла) vs 33 «удобство поиска информации». Матрица рекомендует 1, 17, 32. **Применённые:** #1 «принцип дробления» — матрица встроена как таблица внутри §1 Phase Overview (без новой секции); #17 — front-matter поле agent_routing (другое измерение поверх); #32 — цветное выделение ячеек «✓ native / ✓ wrapper / ✗ / ✓ universal» для мгновенной читаемости.

**Противоречие 3.1.C — «агентопонятность vs объём front-matter».** Если YAML front-matter содержит 10+ полей (invariants, owned_contracts, agent_routing, trace_to_code, ui_budgets), он становится самостоятельным документом, но каждый .md файл утяжеляется. Параметры: 27 «длина» vs 9 «скорость». Матрица рекомендует 1, 2, 7. **Применённые:** #1 «дробление» — front-matter содержит только поля, критичные для маршрутизации агента; #2 «вынесение» — invariant-флаги распределены внутри @invariant тегов в теле документа, не дублируются в front-matter; #7 «антивес» — front-matter весит ~40 строк, но экономит 200+ строк агентового поиска внутри тела.

## 3.2 ROADMAP.md

**Противоречие 3.2.A — «дедупликация vs структурная согласованность».** ROADMAP.md и ARCHITECT.md в исходном состоянии имели идентичную структуру (12 секций). Если ROADMAP.md

остаётся дубликатом, структура согласована, но содержание избыточно. Если ROADMAP.md получает уникальное содержание (продуктовый roadmap-фокус), структура остаётся 12-секционной, но содержание расходится с ARCHITECT. Параметры: 33 «удобство поиска» vs 36 «потери информации» (при расходе версий). Матрица: 10, 17, 32. **Применённые:** #17 — front-matter поле doc: «ROADMAP» явно разделяет слой; #32 — TRIZ-pass changelog block в начале файла указывает, что «содержание дедуплицировано, фокус — deliverables/exit criteria»; #10 «до действия» — Cross-references к ARCHITECT §1 Capability matrix для architectural деталей (агент идёт в ARCHITECT заранее, если нужна architectural layer info).

**Противоречие 3.2.B — «метрики выхода vs тип документа».** ROADMAP — продуктовый документ, но exit criteria должны быть архитектурно обоснованы (capability matrix, fail-safe контракт и т.д.). Параметры: 27 «длина» vs 33 «удобство поиска». **Применённые:** #5 «объединение» — trace-матрица в конце ROADMAP (§12 + новая trace-таблица) связывает цель → ADR → код → UI-секция одним взглядом; #24 «посредник» — ссылка на ARCHITECT §1 для capability matrix вместо дублирования.

## 3.3 UI.md

**Противоречие 3.3.A — «бюджет скорости UI vs полнота описания экрана».** Если каждый экран UI.md несёт свой performance budget в той же секции, документ становится длиннее (~на 1 страницу), но агент находит бюджет сразу. Если бюджет вынесен в §0.3 universal budget, файл компактнее, но агент должен листать к началу файла для конкретного экрана. Параметры: 27 «длина» vs 9 «скорость». Матрица: 1, 10, 32. **Применённые:** #3 «местное качество» — каждый экран имеет §X.6 Performance budget прямо в своей секции (и @invariant тег INV-UI-001…INV-UI-007); #10 «до действия» — skeleton/SWR паттерн (INV-UI-004) применяется ДО того, как данные приедут, обеспечивая sub-100ms first paint даже при медленной загрузке; #32 — color encoding для @invariant критичности (high/critical).

**Противоречие 3.3.B — «универсальный budget vs фазовая адаптивность».** Universal budget (§0.3) один для всех фаз. Но разные фазы предъявляют разные требования (MVP — sub-100ms; BETA — heatmap 200ms; PROD — LLM op 3s). Если budget один — проще поддерживать, но теряется фазовая адаптивность. Параметры: 35 «изменение параметров» (адаптивность) vs 33 «удобство». Матрица: 1, 15, 35. **Применённые:** #1 «дробление» — universal budget в §0.3 + per-screen budgets в §X.6 (два слоя); #15 «динамизация» — ui_budgets_enforced поле в front-matter явно перечисляет все 14 бюджетов с их фазовой привязкой; #35 «фазовый переход» — документирование pattern alignment (Critical CSS analogue, Islands/partial hydration analogue, Skeleton/SWR analogue) как структурных аналогов веб-velocity паттернов.

**Противоречие 3.3.C — «phase conflicts (Dashboard, File tour) vs UI-структура».** ROADMAP §2.1 ставит Dashboard в MVP; UI.md §1.1 ставил Dashboard в ALPHA (до TRIZ). ROADMAP §4.1 ставит File tour в BETA; UI.md §1.1 ставил File tour в PROD. Если UI.md подаётся под ROADMAP, UI-структура теряет самостоятельность; если UI.md остаётся при своих, корпус не согласован. Параметры: 4 «надёжность» (согласованность) vs 33 «удобство». **Применённые:** #4 «асимметрия» — UI.md §1.1 принимает ROADMAP как канон, но добавляет явное «v0.3.1-triz reconciliation» примечание, объясняющее почему (асимметрия: ROADMAP канон для фаз, UI канон для screens).

## 3.4 BACKLOG.md

**Противоречие 3.4.A — «агентопонятность каждого пункта vs объём BACKLOG».** Если каждый backlog-пункт (их 86 active) имеет поле agent_hint с ADR-ссылками, файл растёт ~на 30%, но агент находит связь сразу. Параметры: 27 «длина» vs 33 «удобство». Матрица: 1, 7, 17. **Применённые:** #1 «дробление» — agent_hint — отдельное поле, не слитое с Sketch; #7 «антивес» — agent_hint короткий (1-2 строки), но даёт точный маршрут; #17 «другое измерение» — agent_hint как машинно-читаемое поле (regex-friendly) поверх прозы Sketch.

**Противоречие 3.4.B — «историческая полнота vs текущая актуальность».** BACKLOG должен хранить историю идей (B-004 review TUI, B-005 onboarding, B-006 autocomplete, B-008 color reports, B-009 dashboard, I-007 heatmap, J-003 adapter health — все shipped в v0.3). Если пункты остаются как «idea», BACKLOG вводит агента в заблуждение. Если пункты удаляются, теряется история. Параметры: 27 «длина» vs 4 «надёжность». Матрица: 1, 2, 35. **Применённые:** #2 «вынесение» — статус done вынесен в Status field, не в отдельный «Done» список; #35 «фазовый переход» — Status enum расширен с {idea, draft, proposed, rejected} до {idea, draft, proposed, done, rejected, superseded}; #1 — каждый done-пункт сохранён (история), но с явной плашкой «✅ DONE (v0.3)» и trace к UI/код-контракту.

**Противоречие 3.4.C — «кросс-тематические дубликаты vs полнота тем».** K-003 дублирует B-001 (VSCode), K-004=B-002 (JetBrains), K-005=B-003 (Neovim), K-006=D-006 (voice review). Если дубликаты остаются, BACKLOG загрязнён; если удаляются, темы (UX, AI, UI enhancements) теряют покрытие. Параметры: 33 vs 36. Матрица: 1, 7, 24. **Применённые:** #1 «дробление» — каждый дубликат сохранён в своей теме (K — UI enhancements), но помечен «DUPLICATE of B-NNN» со ссылкой; #7 «антивес» — дубликат сохраняет только Title/Dependencies, без повторного Sketch; #24 «посредник» — trace-матрица в конце BACKLOG указывает каноническую запись для каждой cross-theme дубликатной пары.

## 3.5 ADR.md

**Противоречие 3.5.A — «битые ссылки vs объём ADR».** Если ADR ссылается на ARCHITECT.md §3.3.2/§6.5/§5.2 (4 битые ссылки), агент разочаровывается; если ADR копирует содержание из ARCHITECT, нарушается DRY. Параметры: 4 «надёжность» vs 27 «длина». Матрица: 1, 24, 28. **Применённые:** #1 «дробление» — ссылки исправлены на конкретные секции нового ARCHITECT.md (§1 Capability matrix, §2.5 Risks для fail-safe, §7.4 Backwards compatibility для config); #24 «посредник» — вместо копирования содержания ADR ссылается на конкретный § с описанием; #28 «замена механической системы на электронную/программную» — будущий `caf invariants check` (ADR-029) программно валидирует ссылки.

**Противоречие 3.5.B — «машинная читаемость vs человеческая читаемость».** ADR для людей — прозы. ADR для агентов — структурированные поля. Параметры: 27 «длина» vs 9 «скорость». Матрица: 1, 17, 32. **Применённые:** #1 — @invariant tag как отдельное поле в конце Trigger to reconsider; #17 — @invariant как машинно-читаемое (regex `/@invariant:\s*(.+)/`) поверх прозы Trigger; #32 — INV-ADR-NNN цветная плашка в ADR index таблице.

**Противоречие 3.5.C — «Tier-3 miscount vs поддержание регистра».** Если ADR-012 перечисляет 15 Tier-3 агентов, но помечает «(12)», арифметика «25 explicit» нарушена. Если оставить 15 — count wrong; если убрать 3 — потеря информации. Параметры: 4 «надёжность» vs 36 «потеря информации». Матрица: 2, 22, 35. **Применённые:** #2 «вынесение» — OryaAgent вынесен из Tier-3 (он in-house consumer, описан отдельно в ARCHITECT §6.5); #22 «заблаговременное вредное воздействие» — Augment переименован в

«Augment» (vendor branding, не «Augment Code»), и единообразно между ADR-012 и ROADMAP §5.1 (т.е. ошибки в naming устранены заблаговременно); #35 — будущий ADR-028 (adapter contribution governance) решит вопрос ввода/вывода агентов из регистра.

# Глава 4. Сводная матрица противоречий 39×39

***EN summary.*** *Chapter 4 consolidates the engineering-parameter pairs from the Altshuller 39×39 matrix that emerged in Chapter 3, mapping each pair to its recommended inventive principles and the concrete artifact where each principle was applied.*

Матрица Альтшуллера 39×39 сопоставляет пары инженерных параметров (один «улучшаемый», второй «ухудшаемый») с номерами рекомендованных изобретательских приёмов из 40. Ниже — сводная таблица пар, реально возникших при анализе пяти файлов, с номерами приёмов по матрице и конкретными артефактами-выходами в переписанных файлах. Формула применения: для каждой пары (X, Y) матрица предлагает 3–5 приёмов; в таблице оставлены только те, что реально применены.

| # | Улучшаемый параметр (X) | Ухудшаемый параметр (Y) | Рекомен. приёмы (по матрице) | Применено в файле | Артефакт-выход |
|---|---|---|---|---|---|
| 1 | 33. Удобство поиска информации | 36. Потеря информации (версионный дрейф) | 7, 17, 32 | ARCHITECT.md | YAML front-matter (#17), @invariant (#32), асимметрия содержимого (#7->4) |
| 2 | 27. Длина документа | 33. Удобство поиска | 1, 17, 32 | ARCHITECT.md | Capability matrix как таблица в §1 (#1), front-matter (#17), цветные ячейки matrix (#32) |
| 3 | 27. Длина front-matter | 9. Скорость поиска | 1, 2, 7 | Все 5 файлов | Дробление на поля (#1), вынесение @invariant в тело (#2), антивес (#7) |
| 4 | 33. Удобство поиска (ROADMAP дедупл.) | 36. Потеря информации | 10, 17, 32 | ROADMAP.md | TRIZ-pass changelog (#32), doc: ROADMAP в front-matter (#17), prefetch в ARCHITECT (#10) |
| 5 | 27. Длина UI budget | 9. Скорость поиска | 1, 10, 32 | UI.md | Per-screen §X.6 budgets (#1), skeleton/SWR (#10), @invariant (#32) |
| 6 | 35. Адаптивность фазовая | 33. Удобство | 1, 15, 35 | UI.md | Двухслойный budget (#1), ui_budgets_enforced в front-matter (#15), pattern alignment (#35) |

| # | Улучшаемый параметр (X) | Ухудшаемый параметр (Y) | Рекомен. приёмы (по матрице) | Применено в файле | Артефакт-выход |
|---|---|---|---|---|---|
| 7 | 4. Надёжность (phase conflicts) | 33. Удобство | 4 (асимметрия) | UI.md §1.1 | ROADMAP как канон фаз, UI как канон screens — асимметрия ролей |
| 8 | 27. Длина BACKLOG (с agent_hint) | 33. Удобство | 1, 7, 17 | BACKLOG.md | Дробление agent_hint (#1), антивес короткого hint (#7), машинно-читаемое поле (#17) |
| 9 | 27. Длина (история done) | 4. Надёжность (актуальность) | 1, 2, 35 | BACKLOG.md | Статус done в Status field (#1), вынесение trace (#2), расширенный enum (#35) |
| 10 | 33. Удобство (дубликаты) | 36. Потеря информации | 1, 7, 24 | BACKLOG.md | Дробление (#1), антивес дубликата (#7), trace-матрица-посредник (#24) |
| 11 | 4. Надёжность битых ссылок | 27. Длина (DRY) | 1, 24, 28 | ADR.md | Дробление ссылок на конкретные § (#1), посредник (#24), будущий caf invariants check (#28) |
| 12 | 27. Длина ADR | 9. Скорость (агент) | 1, 17, 32 | ADR.md | @invariant как поле (#1), regex-friendly поверх прозы (#17), цветная плашка (#32) |
| 13 | 4. Надёжность (Tier-3 miscount) | 36. Потеря информации (агент) | 2, 22, 35 | ADR.md | Вынесение OryaAgent (#2), заблаговременное исправление naming (#22), ADR-028 future (#35) |

*Таблица 4.1. Сводная матрица противоречий 39×39 (13 пар, применённых в переписанных файлах).*

Из 13 пар 4 пары — «надёжность vs длина» (4, 9, 11, 13), что отражает центральную проблему корпуса: документация либо короткая и согласованно-битая, либо длинная и согласованно-полная. ТРИЗ-приём #1 «дробление» — самый частый ответ (8 из 13 пар), что методологически оправдано: каждое противоречие «длина vs что-то» лучше всего снимается разделением длинного объекта на части с разными свойствами. Второй по частоте — #17 «переход в другое измерение» (5 пар), что соответствует добавлению YAML front-matter как машинно-читаемому слою поверх прозы.

# Глава 5. Подобранные приёмы из 40 — маппинг на артефакты

***EN summary.*** *Chapter 5 maps the inventive principles (from the 40) actually applied in the rewritten files to their concrete artifact — what new field, what new table, what new tag, what new cross-reference each principle produced. 14 principles applied; each entry includes the principle name, where it is applied (file/section), and the resulting artifact in the rewritten corpus.*

Из 40 приёмов Альтшуллера в переписанных файлах применены 14. Каждый приём произвёл конкретный артефакт — новое поле, новую таблицу, новый тег, новую cross-reference. Ниже — таблица «приём → артефакт».

| # | Название приёма | Где применяется | Артефакт в переписанных файлах |
|---|---|---|---|
| 1 | Принцип дробления | Все 5 файлов | YAML front-matter как отдельный слой (5 полей: layer/version/invariants/owned_contracts/agent_routing); capability matrix как таблица внутри §1 ARCHITECT; per-screen §X.6 budgets в UI.md |
| 2 | Принцип вынесения | ADR.md, BACKLOG.md, ARCHITECT.md | @invariant теги вынесены в Trigger field ADR (не в front-matter); Status done в BACKLOG Status field (не в отдельный список); OryaAgent вынесен из Tier-3 в ARCHITECT §6.5 |
| 3 | Местное качество | UI.md, BACKLOG.md | Каждый экран несёт свой performance budget в своей §X.6 (не в общем §0.3); каждый backlog-пункт несёт свой agent_hint (не в общем §0) |
| 4 | Асимметрия | ARCHITECT.md vs ROADMAP.md, UI.md vs ROADMAP.md | ARCHITECT и ROADMAP дифференцированы по содержанию (architecture vs product), сохраняя 12-секционную структуру; ROADMAP — канон фаз, UI — канон screens (асимметрия ролей) |
| 5 | Объединение | BACKLOG.md, ROADMAP.md | Trace-матрица в конце BACKLOG связывает Цель → ADR → backlog → код-контракт одним взглядом; ROADMAP §12 + trace-таблица — тот же паттерн |
| 6 | Универсальность | Все 5 файлов | YAML front-matter с единым каркасом (doc/layer/version/language/status/audience/related/depends_on/invariants/owned_contracts/agent_routing/trace_to_code/ui_budgets) — единая header schema для всех .md |
| 7 | Антивес (Mario cart) | BACKLOG.md, ADR.md | agent_hint короткий (1-2 строки), но даёт точный маршрут; дубликаты K-003..K-006 сохранены с минимальным телом (Title + Dependencies), без повторного Sketch |
| 8 | Анти-вес (Counterweight) | ARCHITECT.md | Fail-safe контракт (§2.5) — тяжёлый counterweight против «оптимизаций», которые ломают INV-ARCH-004 fail-safe behavior |
| 9 | Предварительное анти-вредное воздействие | ADR.md (ADR-012) | Augment переименован в «Augment» (vendor branding) единообразно между ADR-012 и ROADMAP §5.1 — устранён будущий naming drift |
| 10 | До действия | UI.md | Skeleton/SWR паттерн (INV-UI-004): Skeleton показывается ДО того, как данные приедут, обеспечивая sub-100ms first paint; prefetch в ARCHITECT.md §1 для capability matrix |
| 13 | Инверсия | BACKLOG.md | Trace-матрица: вместо «от цели к коду» (forward) — bidirectional trace «код → ADR → backlog → цель» (агент приходит от кода, не от цели) |
| 15 | Динамизация | UI.md | ui_budgets_enforced поле в front-matter явно перечисляет все 14 бюджетов с фазовой привязкой (бюджеты меняются по фазам, не статичны) |

| # | Название приёма | Где применяется | Артефакт в переписанных файлах |
|---|---|---|---|
| 17 | Переход в другое измерение | Все 5 файлов | YAML front-matter как машинно-читаемое измерение поверх прозы; @invariant теги как regex-friendly поверх Trigger field; agent_hint как parseable поле поверх Sketch |
| 24 | Посредник | BACKLOG.md, ADR.md | Trace-матрица-посредник между cross-theme дубликатами (K-003 ↔ B-001); ADR ссылается на ARCHITECT §X.Y через посредник-секцию (не копирует содержание) |
| 25 | Самообслуживание | ARCHITECT.md | caf doctor self-healing — .caf/ регенерируется из YAML source of truth (storage layer сам себя чинит) |
| 26 | Копирование | UI.md | TUI стек реализует структурные аналоги веб-velocity паттернов (Critical CSS, Islands/partial hydration, Skeleton/SWR) — копирование паттернов между платформами |
| 32 | Оптические изменения | Все 5 файлов | Цветные @invariant теги; цветные ✓/✗ в capability matrix; цветные плашки INV-ADR-NNN в ADR index |
| 35 | Изменение параметров | UI.md, BACKLOG.md | UI-бюджеты меняются по фазам (MVP sub-100ms, BETA heatmap 200ms, PROD LLM 3s); Status enum расширен с 4 до 6 значений (добавлены done, superseded) |
| 36 | Фазовый переход | ARCHITECT.md §6.3, UI.md §1.4 | TUI→Web фазовый переход при FUTURE (только для visualizations impossible in TUI); ROADMAP §6.3 Web dashboard явно как фазовый переход, не как замена TUI |

*Таблица 5.1. Маппинг 18 приёмов из 40 (применённых в переписанных файлах) → конкретные артефакты-выходы.*

Наиболее продуктивные приёмы — #1 (применён в 8 местах), #17 (5 мест), #32 (5 мест), #2 (3 места). Совокупно они формируют «технологический платформенный» эффект: дробление + переход в другое измерение + оптические изменения + вынесение = новая архитектура документации, где каждый файл имеет два слоя (машинно-читаемый front-matter + человеческая проза), каждый инвариант помечен, каждая cross-reference посредничает. Это и есть структурное решение ИКР.

# Глава 6. АРИЗ-шаги для двух ключевых проблем

***EN summary.*** *Chapter 6 runs the full ARIZ-85-V procedure (Algorithm for Inventive Problem Solving) for the two most acute problems identified in the digest: (1) ARCHITECT.md as a byte-identical duplicate of ROADMAP.md (and the resulting absence of the capability matrix); (2) BACKLOG.md stale items + cross-theme duplicates + broken §6.10 cross-references. For each problem, the seven ARIZ steps are documented: mini-problem statement, conflicting pairs, conflict zone, IFR-2 formulation, application of principles, evaluation, control solution.*

## 6.1 АРИЗ-прогон для проблемы №1: ARCHITECT.md = дубликат ROADMAP.md

### Шаг 1.1. Мини-задача

Сформулировать минимально необходимое изменение, которое устраняет дублирование ARCHITECT.md и ROADMAP.md, не нарушая 12-секционную структуру файлов (по требованию пользователя «только содержание») и не удаляя ни одного существующего заголовка. Мини-задача: переписать содержание ARCHITECT.md в архитектурном ключе (architectural layers, invariants, capability matrix, fail-safe contract, config schema), сохранив ту же 12-секционную структуру и тот же H1 pattern. ROADMAP.md остаётся продуктовым документом (deliverables, exit criteria, milestones).

### Шаг 1.2. Конфликтующие пары

Пара 1: «содержание ARCHITECT должно быть уникальным» (удобство поиска 33 ↑) vs «структура 12 секций неизменна» (надёжность 4 ↓ — нарушить структуру = потерять согласованность с ROADMAP).

Пара 2: «capability matrix должна быть в ARCHITECT» (33 ↑) vs «объём ARCHITECT растёт» (27 ↓).

Пара 3: «4 битые ссылки из ADR на ARCHITECT §3.3.2/§6.5/§5.2 должны быть исправлены» (4 ↑) vs «DRY — копировать содержание в ADR нельзя» (27 ↓).

### Шаг 1.3. Зона конфликта

Зона конфликта — первый экран каждого .md файла. Именно туда агент попадает первым; именно там (в исходном состоянии) его встречает неуникальное содержание и неопределённые ссылки. Решение должно быть реализовано в пределах front-matter + первых 2 секций каждого файла, чтобы агент получал маршрутизацию с первого экрана.

### Шаг 1.4. ИКР-2 формулировка

ИКР-2: «YAML front-matter сам по себе, без участия агента-человека, предоставляет маршрут к capability matrix, fail-safe контракту и config schema; агент, открыв ARCHITECT.md, в первом экране находит все три ссылки на конкретные §, а внутри этих § — соответствующее содержание в виде таблиц или вложенных блоков (не внешних ссылок)». ИКР-2 формализует требование «без дополнительных инструментов» на уровне front-matter.

### Шаг 1.5. Применение приёмов

#1 «дробление» — YAML front-matter разделён на 12 полей, из которых 4 (invariants, owned_contracts, agent_routing, trace_to_code) предназначены специально для агентов.

#4 «асимметрия» — ARCHITECT и ROADMAP сохраняют 12-секционную структуру, но содержание дифференцировано (asymmetry by content).

#17 «переход в другое измерение» — YAML front-matter — отдельное машинно-читаемое измерение поверх прозы.

#24 «посредник» — ADR ссылается на конкретные § ARCHITECT (§1 Capability matrix, §2.5 Risks для fail-safe, §7.4 Backwards compatibility для config) через посредника-секцию, не копируя содержание.

### Шаг 1.6. Оценка полученных решений

Все четыре применённых приёма согласованы и не создают новых противоречий: #1 + #17 — front-matter становится одновременно структурированным (#1) и машинно-читаемым (#17); #4 — ROADMAP остаётся в той же структуре, что и ARCHITECT (симметрия сохранена); #24 — ADR не

копирует содержание ARCHITECT, но ссылается точно (DRY сохранён). Новых противоречий не выявлено.

### Шаг 1.7. Контрольное решение — конкретные изменения

В новом ARCHITECT.md: (a) H1 исправлен с «# CodeAuthorityFirst — ROADMAP.md» на «# CodeAuthorityFirst — ARCHITECT.md»; (b) добавлен YAML front-matter (60 строк) со всеми 12 полями, включая 7 @invariant тегов (INV-ARCH-001…INV-ARCH-007); (c) §1 Phase Overview дополнен таблицей Capability matrix (28 строк, 25 explicit + universal); (d) §2.5 Risks дополнен вложенным блоком «Fail-safe contract» (4-пунктовый контракт, включая exit codes); (e) §7.4 Backwards compatibility дополнен Config schema (15+ ключей .caf/config.yml). В новом ADR.md: 4 битые ссылки исправлены — ADR-003 ссылается на ARCHITECT.md §2.5 Risks (вместо несуществующего §6.5); ADR-004 на §7.4 (вместо §5.2); ADR-009 и ADR-013 на §1 Capability matrix (вместо несуществующего §3.3.2).

## 6.2 АРИЗ-прогон для проблемы №2: BACKLOG.md устаревшие пункты + дубликаты + битые ссылки

### Шаг 2.1. Мини-задача

Переписать BACKLOG.md так, чтобы: (a) 7 пунктов, уже shipped в v0.3 (B-004, B-005, B-006, B-008, B-009, I-007, J-003), были явно помечены done; (b) 4 кросс-тематических дубликата (K-003=B-001, K-004=B-002, K-005=B-003, K-006=D-006) были явно отмечены как дубликаты; (c) 3 битые ссылки на ROADMAP §6.10 были исправлены на §6.9; (d) статистика была пересчитана; (e) каждый backlog-пункт имел agent_hint с ADR-ссылками. Сохранить 16-секционную структуру файла.

### Шаг 2.2. Конфликтующие пары

Пара 1: «история сохранена (B-004 и т.д. остаются)» (36 ↑ — потеря информации предотвращена) vs «актуальность (B-004 помечен done)» (4 ↓ — надёжность требует, чтобы статус был явным).

Пара 2: «кросс-тематические дубликаты сохранены (K-003 и т.д.)» (33 ↑ — полное покрытие тем) vs «не загрязняют» (4 ↓ — надёжность требует, чтобы не было ложных независимых пунктов).

Пара 3: «каждый пункт имеет agent_hint» (33 ↑ — удобство агента) vs «BACKLOG растёт» (27 ↓ — объём файла).

### Шаг 2.3. Зона конфликта

Зона конфликта — каждое поле §X.Y backlog-пункта (Theme, Status, Problem, Sketch, Open questions, Dependencies, agent_hint). Именно здесь должны быть сделаны изменения, не выходящие за пределы существующего шаблона.

### Шаг 2.4. ИКР-2 формулировка

ИКР-2: «Status field сам по себе различает 6 состояний (idea/draft/proposed/done/rejected/superseded) без необходимости отдельного списка done-пунктов; agent_hint поле в каждой записи даёт точный маршрут к ADR и код-контракту; дубликаты сохранены с минимальным телом (Title + DUPLICATE note +Dependencies), не раздувая файл». ИКР-2 требует, чтобы все три проблемы решались в пределах существующего шаблона пункта, без добавления новых полей (кроме agent_hint).

### Шаг 2.5. Применение приёмов

#1 «дробление» — Status enum расширен (добавлены done, superseded); agent_hint — отдельное поле, не слитое со Sketch.

#2 «вынесение» — trace done-пунктов вынесен в Status field (не в отдельный список); trace-матрица вынесена в §17 (не в каждый пункт).

#7 «антивес» — дубликаты K-003..K-006 сохранены, но с минимальным телом (Title + DUPLICATE note +Dependencies), без повторного Sketch.

#24 «посредник» — trace-матрица в §16 служит посредником между cross-theme дубликатами (K-003 ↔ B-001 и т.д.).

#35 «фазовый переход» — Status enum расширен с 4 до 6 значений (фазовый переход статуса).

### Шаг 2.6. Оценка полученных решений

Все 5 приёмов согласованы: #1 + #35 — Status enum расширен, не создавая нового поля; #2 — trace вынесен в §17; #7 — дубликаты минимальны; #24 — посредник-матрица работает. Новых противоречий нет: BACKLOG.md v0.3.1-triz на ~30% длиннее оригинала (за счёт agent_hint в 86 пунктах и trace-матрицы в §17), но это компенсируется сокращением дубликатного содержания (K-003..K-006 не дублируют Sketch) и явным маркированием done (агент не тратит время на уже-shipped пункты).

### Шаг 2.7. Контрольное решение — конкретные изменения

В новом BACKLOG.md: (a) B-004, B-005, B-006, B-008, B-009, I-007, J-003 — все 7 помечены «✅ DONE (v0.3)» с указанием UI-секции, где это реализовано; (b) K-003, K-004, K-005, K-006 — все 4 помечены «DUPLICATE of B-NNN/D-NNN» с явной плашкой «kept for theme completeness»; (c) §6.10 → §6.9 в трёх местах (§0 Backlog Philosophy, §13 Rejected Items prose, §14 How to Promote); (d) статистика в §16 пересчитана: proposed=15 (было 16), draft=10 (было 15), idea=44 (было 51), done=7 (новое поле), superseded=1 (новое поле для E-007). Арифметика согласована; (e) каждый из 86 active пунктов имеет поле agent_hint с прямыми ссылками на ADR и код-контракт; (f) K-012 typo `aria2` → WAI-ARIA; (g) D-006 китайский символ «语音» → русское «голосом»; (h) D-008 китайский «合成» → русское «синтез».

# Глава 7. Резюме улучшений и trace-матрица

***EN summary.*** *Chapter 7 summarises the before→after changes per file and consolidates the traceability matrix linking each architectural goal to its ADR, backlog item, code contract, and UI section.*

## 7.1 Было → стало по 5 файлам

| Файл | Было (v0.3.0) | Стало (v0.3.1-triz) |
|---|---|---|
| ARCHITECT.md | Байт-идентичен ROADMAP.md (H1 «ROADMAP.md»); 4 битые ссылки из ADR; capability matrix не существует; fail-safe контракт не описан; config schema не описана | H1 исправлен на «ARCHITECT.md»; YAML front-matter (7 invariants INV-ARCH-001…007); §1 Capability matrix (28 строк); §2.5 Fail-safe contract (4-пунктовый); §7.4 Config schema (15+ ключей) |

| Файл | Было (v0.3.0) | Стало (v0.3.1-triz) |
|---|---|---|
| ROADMAP.md | Байт-идентичен ARCHITECT.md; содержание — roadmap в файле с именем ARCHITECT | YAML front-matter (doc: ROADMAP); содержание фокусировано на deliverables/exit criteria; §1 ссылается на ARCHITECT §1 для capability matrix; §6.9 — канон для 4 promotion criteria (исправлены 3 битые ссылки из BACKLOG) |
| UI.md | 15 секций; §1.1 ставил Dashboard=ALPHA (vs ROADMAP MVP), File tour=PROD (vs ROADMAP BETA); §0.2 TUI stack без явных velocity-pattern аналогов; performance budgets без @invariant тегов; front-matter ссылается только на ARCHITECT+ROADMAP (нет BACKLOG/ADR) | 15 секций сохранены; §1.1 Dashboard→MVP, File tour→BETA (reconciliation с ROADMAP); §0.2 явные 3 velocity-pattern аналога (Critical CSS / Islands / Skeleton-SWR); каждый §X.6 Performance budget имеет @invariant тег INV-UI-001…007; front-matter ссылается на все 4 sibling файла |
| BACKLOG.md | 16 секций; 7 shipped пунктов помечены как idea/draft/proposed; 4 cross-theme дубликата (K-003/4/5/6); 3 битые ссылки §6.10; статистика не сходится (16 vs 19 proposed, 15 vs 16 draft, 82 vs 86 total); K-012 typo `aria2`; D-006 китайский «语音»; D-008 китайский «合成»; нет agent_hint полей | 16 секций сохранены; 7 shipped помечены ✓ DONE; 4 дубликата явно отмечены DUPLICATE; §6.10→§6.9 (3 места); статистика пересчитана (15/10/44/7/1/7); K-012→WAI-ARIA; D-006→«голосом»; D-008→«синтез»; каждый из 86 пунктов имеет agent_hint; новая §17 Trace Matrix |
| ADR.md | 15 ADR + §0 Format + §16 How to Add + §17 ADR Backlog; 4 битые ссылки на ARCHITECT §3.3.2/§6.5/§5.2; Tier-3 miscount (15 listed vs «12» label); Augment Code vs Augment naming drift; ADR-005 stale v0.2 string в trigger; все 15 ADR dated 2026-10-28 | 15 ADR + §0 Format + §16 How to Add + §17 ADR Backlog (та же структура); 4 битые ссылки исправлены на §1/§2.5/§7.4; Tier-3 reconciled (12 actual, OryaAgent removed); Augment consistently; ADR-005 trigger переписан под v0.3; каждый ADR имеет @invariant тег; ADR-029 (новый) предложен для caf invariants check |

*Таблица 7.1. Было → стало по 5 файлам (v0.3.0 → v0.3.1-triz).*

## 7.2 Trace-матрица: Цель → ADR → Backlog → Код → UI

Trace-матрица (явный артефакт ТРИЗ-приёма #5 «объединение» и #13 «инверсия») связывает пять слоёв документации в один взгляд: архитектурная цель → ADR (Decision Record) → backlog item → код-контракт → UI-секция. Это и есть материализация под-ИКР «без дополнительных инструментов» — теперь агент, открыв BACKLOG.md §17 или ARCHITECT.md Trace Matrix, видит все связи одним взглядом.

| Цель | ADR | Backlog item | Код-контракт | UI-секция |
|---|---|---|---|---|
| Provenance by default | ADR-001, ADR-003 | (implemented) | internal/caf/storage/, internal/caf/hooks/ | UI.md §3.5 Refresh strategy |
| SRS-based mental model | ADR-002 | D-007 (adaptive difficulty) | internal/caf/srs/sm2.go | UI.md §4 Review Session |
| Triviality threshold | ADR-004 | H-006 (Lua rules) | internal/caf/triviality/ | (internal) |
| Adapter pattern | ADR-009, ADR-012 | H-002 (custom adapters), E-007 (superseded) | internal/caf/adapter.go, internal/adapters/ | UI.md §7 Adapters Manager |

| Цель | ADR | Backlog item | Код-контракт | UI-секция |
|---|---|---|---|---|
| Universal LSP fallback | ADR-013 | E-007 (superseded by ADR-013) | internal/caf/lsp/fallback.go | UI.md §7.4 Health check view |
| TUI-first UI | ADR-014 | K-001..K-012 (UI enhancements) | pkg/ui/ | UI.md §0–§15 |
| 80/20 prioritization | ADR-015 | 8 high-leverage features in ROADMAP §0.1 | (cross-cutting) | UI.md §0.3 Performance budget |
| Fail-safe hook execution | ADR-003 (INV-ADR-003), ADR-010 | (implemented) | internal/caf/hooks/fail_safe.go | (internal — described in ARCHITECT §2.5) |
| Invariant enforcement (proposed) | ADR-029 (proposed) | (BACKLOG §0 template — agent_hint fields) | internal/caf/invariants/ (future) | — |
| Decision Log layer | ADR-001, ADR-005 | (implemented in ALPHA) | internal/caf/decisions/ | UI.md §5.1–5.3 |
| MCP server + LLM narratives | ADR-006, (ADR-017 future) | D-001 (LLM pre-fill), D-008 (multi-session narratives) | internal/caf/mcp/server.go | (server-side, no TUI) |
| Multi-language graph adapters | ADR-011 | E-001..E-008 | internal/caf/lang/ | (no UI in MVP) |

*Таблица 7.2. Trace-матрица: 13 архитектурных целей × 5 слоёв документации.*

Trace-матрица — единственный артефакт, которого не было в исходном корпусе и который ТРИЗ-проход добавил явно в ARCHITECT.md (Trace Matrix в конце файла) и в BACKLOG.md (§17 Trace Matrix). Это материализация приёма #5 «объединение»: вместо пяти разрозненных файлов — один сводный взгляд.

# Глава 8. Ограничения, компромиссы и следующие шаги

***EN summary.*** *Chapter 8 documents what TRIZ did not solve, the trade-offs accepted in the rewritten files, and recommended next steps for the user.*

## 8.1 Что ТРИЗ не решил

Во-первых, будущее дублирование при эволюции корпуса. ТРИЗ一次性 устранил существующие дубликаты, но не предотвратил будущие. Если мейнтейнер добавит новую секцию в ARCHITECT.md и забудет синхронизировать ROADMAP.md, дублирование вернётся. Решение — будущий `caf doc-check` pre-commit hook (ADR-029, предложен в TRIZ-проходе), который программно валидирует cross-references и инварианты. Это задача для отдельной итерации.

Во-вторых, баланс bilingual vs single-language. Пользователь выбрал билингву (.md на английском как international OSS, отчёт на русском). Это создаёт дрейф: русские summary в каждом файле должны обновляться синхронно с английским телом. YAML front-matter остаётся EN-only как машинно-читаемый слой — это снижает дрейф (front-matter не нужно переводить), но не устраняет его полностью. Долгосрочное решение — RFC на bilingual maintenance process, выходящий за рамки TRIZ-прохода.

В-третьих, устойчивость @invariant флагов без парсера. ТРИЗ-проход добавил 22 @invariant тега (7 в ARCHITECT, 7 в UI, 3 в BACKLOG, 5 в ADR), но без программной валидации это просто текст. Если мейнтейнер нарушит инвариант в коде, ничего не сломается — пока. Решение — будущий `caf invariants check` (ADR-029), который будет парсить @invariant теги и проверять их programmatically. До реализации ADR-029 @invariant — это конвенция, а не enforcement.

## 8.2 Компромиссы, принятые в переписанных файлах

Добавление YAML front-matter и agent_hint полей увеличило объём каждого .md файла на ~15-30%. Совокупно корпус вырос с ~243 KB (5 файлов в исходном состоянии, без дублирования ARCHITECT=ROADMAP) до ~290 KB (после TRIZ-прохода). Это компенсируется: (a) устранением дублирования (ROADMAP теперь не копия ARCHITECT — экономия ~31 KB); (b) явным маркированием done-пунктов (агент не тратит время на уже-shipped); (c) trace-матрицами, которые сокращают agent-навигацию с 5 шагов до 1. Чистый эффект — рост корпуса на ~19%, но сокращение средней длины agent-trajectory от ~5 файлов до ~1.5.

Второй компромисс — сохранение 12-секционной структуры ARCHITECT.md и ROADMAP.md (по требованию пользователя «только содержание»). Это означает, что оба файла выглядят структурно идентично, и агент должен читать front-matter (поле `doc:`), чтобы различить их. Если бы реструктуризация была разрешена, ARCHITECT.md получил бы иную структуру (например, layer-by-layer вместо phase-by-phase), и дифференциация была бы визуальной. В рамках данного TRIZ-прохода принято решение следовать user constraint и компенсировать структурную симметрию содержательной асимметрией (приём #4).

## 8.3 Следующие шаги для пользователя

**1. Просмотреть переписанные файлы.** Все 5 .md файлов сохранены в /home/z/my-project/download/. Рекомендуется сделать git diff между исходными файлами (analytics/ARCHITECT.md и т.д.) и новыми (download/ARCHITECT.md и т.д.) для review. Особое внимание — на ADR.md (4 битые ссылки исправлены, Tier-3 reconciled) и BACKLOG.md (статистика пересчитана, 7 пунктов помечены done).

**2. Внедрить caf doc-check pre-commit hook (FUTURE ADR-029).** Это программная валидация cross-references и @invariant тегов. Без этого hook-а инварианты остаются конвенцией, а не enforcement. Предлагается реализовать как часть MVP/ALPHA: парсер markdown → проверка всех ссылок вида `[ADR-NNN](./ADR.md#adr-NNN-...)` → проверка, что якорь существует. Это устранит будущее дублирование.

**3. Пилот на репо kubernetes/kubernetes.** ROADMAP §4.5 BETA risk указывает, что graph builder должен benchmark-нуться на kubernetes/kubernetes (>10k файлов). В контексте TRIZ-прохода рекомендуется также валидировать agent_routing поле ARCHITECT.md на реальной задаче агента —

например, «add new adapter for agent X» — и убедиться, что front-matter даёт достаточный маршрут без чтения всего файла.

**4. Через 4 недели — переоценка метрик.** Цель TRIZ-прохода — поднять агентопонятность, что должно проявиться в стабилизации ACS, Recall, Decision Coverage на pilot repos. Если через 4 недели после внедрения переписанных файлов метрики улучшатся (или хотя бы не регрессируют), TRIZ-проход можно считать успешным. Если регрессируют — вернуться к §6.1 Шаг 1.6 (оценка применённых приёмов) и пересмотреть.

## 8.4 Итог

TRIZ-проход переписал 5 файлов, исправил 15 инконсистентностей, добавил 7 @invariant тегов в ARCHITECT.md, 7 в UI.md, 22 совокупно во всём корпусе, и 2 trace-матрицы (в ARCHITECT и BACKLOG). Применены 18 из 40 приёмов Альтшуллера; наиболее продуктивные — #1 (дробление, 8 мест), #17 (переход в другое измерение, 5 мест), #32 (оптические изменения, 5 мест). Два полных АРИЗ-прогона (Главы 6.1 и 6.2) документируют, как приёмы разрешают конкретные противоречия. ИКР «документация сама себя объясняет LLM-агенту за один проход без дополнительных инструментов» частично материализован через YAML front-matter (машинно-читаемый слой), @invariant теги (regex-friendly) и trace-матрицы (объединение 5 слоёв в один взгляд). Полная материализация ИКР ожидается после реализации ADR-029 (caf invariants check), предложенного в этом TRIZ-проходе.